user_id	user_name	city
3	Anele	Durban

Eg 2

membership_plan

plan_id	plan_name	monthly_pri
101	Basic	79
102	standa	129
103	premi	199
104	famil	249
105	mobile	59

Write an SQL query to give me the plans that cost more than 150

NB

```
SELECT plan_name
       monthly_price
FROM   membership_plan
WHERE  monthly_price > 150;
```

Select does not really affect * take all this are specified

plan_name	monthly_price
Premium	199
Family	249

SQL logical Operators

AND statement

- allows us to combine multiple conditions
- All the conditions must be satisfied (met)
- We commonly use the AND statement when building logical conditions. for filtering our data

Select colum 1
colum 2

-

Students

student_id	name	Course	Age	City
1	Thabo	Data Ana	22	Jhb
2	Sarah	Saen	19	Preter
3	John	Ana	25	Durb
4	Lerato	Ana	20	JHB
5	Musa	Sae	23	JHB

Scanned with CamScanner

Write an SQL query to give me the names of students living in JHB that are doing Andly

```
SELECT *
FROM Students
WHERE city = 'JHB'
AND course = 'Data Andly'
```

Student id	name	course	age	city
1	Thabo	Andly	22	JHB
2	Lerato	Data	20	JHB

OR Statement

- combines multiple conditions one of the condition must be met

Same table as above

Write an SQL query to give me the names of ~~students~~ students living in Johannesburg or the students doing Data Analytics

```
SELECT *
FROM students
WHERE course = 'Data Analytic
      OR City = 'Jchans
```

Student id	Name	course	age	City
1	Thabo			
3	John			
4	Lerato			
5	musa			

IN Statement

It is a way of filtering data by specifying the values of interest from a specific column

Syntax:

```
SELECT *
FROM table_name
WHERE Column IN
('value', 'value', 'value
.....)
```

no limit number of values you can specify

write an SQL query to give me people living in Durban and Pretoria

```
SELECT name,
       city
FROM Students
WHERE city IN ('Durban',
'Pretoria')
```

name	city
Sarah	Pretoria
John	Durban

NOT Statement
L allows us to get the opposite result.
L negative result
L exclusion statement

name	city
Thabo	
Lerato	
musa	

SELECT name,
city
FROM ~~stude~~
WHERE ~~not~~ city not in
('Durban', 'Pretoria')

BETWEEN STANTENT

- It allows us to filter the data based on a specific range of values
- It is an inclusive statement. This means that the lower and upper bounds of the Between statement are include

AND and BETWEEN do hand in hand

SELECT: column 1
column 2
column 3
From table_name
WHERE column BETWEEN lower bound AND upper bound

Write an SQL query to give the name & students the ages of 20 to 28

```
SELECT name,
       age
FROM students
WHERE age BETWEEN
20 AND 23
```

name	age
Thabo	22
Lerato	20
Musa	23

SORTING STATEMENT

- A way to sort our data in different ways

ORDER BY STATEMENT

- In SQL this is a sorting statement
- We can sort our data either in Ascending Order (ASC) or Descending order (DESC)
- Sorting does not remove or add anything new into our data - it just rearrange the rows in our data

rearranging data

Ascending order - Its going up
- from smallest to biggest
- lowest to highest
1-9
9-Z

Scanned with CamScanner

Descending order → its going down

(DESC)

9-1 - biggest to smaller

2-9 < highest to lowest

Ascending Order Syntax:

```
SELECT *
FROM table_name
WHERE condition
ORDER BY column_name
ASC;
```

Descending Order Syntax:

```
SELECT *
FROM table_name
WHERE condition
ORDER BY column_name
DSC
```

SAME TABLE

HSC

```
SELECT name,
       age
FROM students
ORDER BY age ASC;
```

name	age
Sarah	19
Lerato	20
Thabo	22
Musa	23
John	25

DSC

```
SELECT name,
       age
FROM students
ORDER BY age DESC;
```

name	age
John	25
Musa	23
Thabo	22
Lerato	20
Sarah	19

Scanned with CamScanner

Aggregate Functions

- An aggregate function is used to perform a calculation on rows of a specific column and returns a single value / a single value per group

Functions	Description
1. Count()	Counts the number of rows
2. Sum()	Calculates the table of a numeric column
3. Average AVG()	Calculates the average of a numeric column
4. MAX()	Find the maximum value
5. MIN()	Find the minimum value

Employees

emp_id	Name	Salary
101	John	20
102	Mary	40
103	David	35

Scanned with CamScanner

Min function

```
SELECT MIN(Salary) As
           min_salary
FROM       employee
```

min_salary
20

Max Function

```
SELECT max(salary) AS
           max_salary
FROM       employee;
```

max_sala.
40

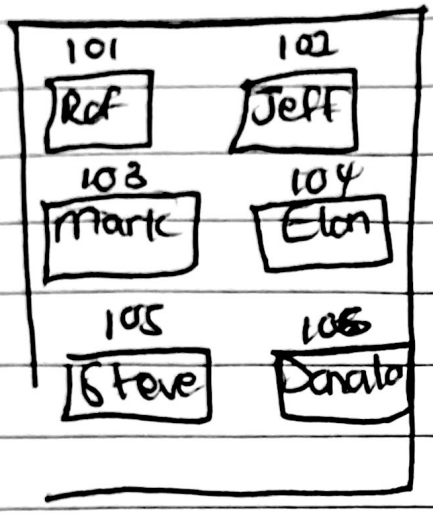
AS

↳ A way to rename a column after applying of function

COUNT()

6

Grade 10H

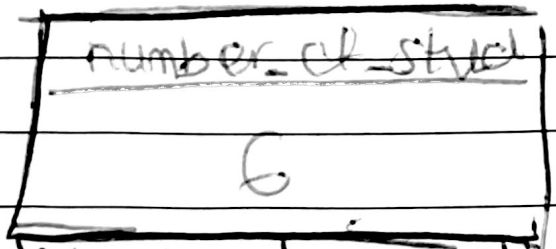


Students

Student_id	Name	Subject
101	Rofhi	Scie
102	Jeff	Commerc
103	Mark	Commerc
104	Elon	Scie
105	Steve	Histo
106	Donal	Commerc

Select Count (Distinct student_id)
As number_of_students
From students

Counting the number of students in the class using their ID



We count something that is uniquely identified like an id

~~(Record Roll Student)~~

- We dont count the revenue, we sum() the revenue
- we do not sum an ID, rather we count an ID

eg: count ID
sum marks

GROUP BY STATEMENT

↳ A statement that allows us to group our data into categories

Students table

eg. Subject is a categorical
Subject = categorical
= not aggregate

```
SELECT COUNT(DISTINCT column
Student_id) AS number_of_students
FROM students
GROUP BY subject;
```

aggregate column — not aggregate

~~that is~~

- we group by a column that is not aggregated we commonly group by a categorical column

Subject	number_of_students
Sale	2
Commerce	3
History	1

Eg.

Sales

Sales_id	Customer_name	Cate	Rev
1	John	Electron	120
2	Mary	Food	80
3	~~Robert~~ Elon	Electron	150
4	Peter	Clothi	100

Write an SQL query to calculate the total revenue for each category.

```
SELECT  Category, Sum(revenue) AS total_revenue
From    Sales
GROUP BY Category
```

no clarity because category revenue diff

Category	Total Rev
elec	270
Food	80
clothing	100

Scanned with CamScanner